CITED REFERENCES AND FURTHER READING:

Dennis, J.E., and Schnabel, R.B. 1983, *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*; reprinted 1996 (Philadelphia: S.I.A.M.).[1]

Jacobs, D.A.H. (ed.) 1977, *The State of the Art in Numerical Analysis* (London: Academic Press), Chapter III.1, §3 – §6 (by K. W. Brodlie).[2]

Polak, E. 1971, *Computational Methods in Optimization* (New York: Academic Press), pp. 56ff.[3]

Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), pp. 467–468.

10.10 Linear Programming: The Simplex Method

The subject of *linear programming*, sometimes called *linear optimization*, concerns itself with the following problem: For n independent variables $x_1, \ldots, x_n$, *minimize* the function

$$\zeta = c_1 x_1 + c_2 x_2 + \cdots + c_n x_n \tag{10.10.1}$$

subject to the nonnegativity conditions

$$x_1 \geq 0, \quad x_2 \geq 0, \quad \ldots \quad x_n \geq 0 \tag{10.10.2}$$

and simultaneously subject to m additional constraints of the form

$$a_{i1} x_1 + a_{i2} x_2 + \cdots + a_{in} x_n \leq b_i \tag{10.10.3}$$

or

$$a_{i1} x_1 + a_{i2} x_2 + \cdots + a_{in} x_n = b_i \tag{10.10.4}$$

Here $i = 1, \ldots, m$. Note that an inequality with a $\geq$ can be converted to a $\leq$ by multiplying by -1. Some formulations of linear programming require you to write all the constraints with the b's nonnegative and separately treat the $\geq$ and $\leq$ constraints. We will use the above formulation, with either sign of b_i, instead. However, it is still useful to refer to the inequalities with $b_i \leq 0$ as "$\geq$ inequalities" (which they would be with $b_i \geq 0$), since, as we shall see, they enter the problem in a different way from the $\leq$ inequalities.

There is no particular significance in the number of constraints m being less than, equal to, or greater than the number of unknowns n. Also, note that there is no special significance to minimizing ζ in equation (10.10.1): We can convert a maximizing problem to a minimizing problem by changing the signs of all the c's. The solution $x_1, \ldots, x_n$ is the same, and the required maximum is the negative of the minimum ζ found.

A set of values $x_1, \ldots, x_n$ that satisfies the constraints (10.10.2) – (10.10.4) is called a *feasible vector*. The function that we are trying to minimize is called the *objective function*. The feasible vector that minimizes the objective function is called the *optimal feasible vector*. An optimal feasible vector can fail to exist for two distinct reasons: (i) There are *no* feasible vectors, i.e., the given constraints

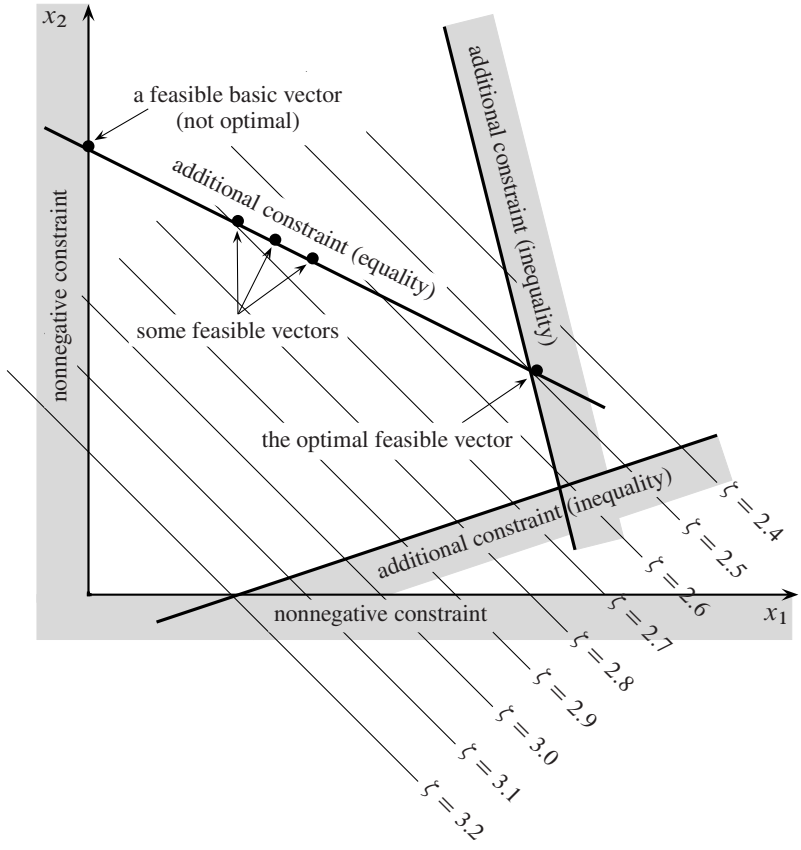

Figure 10.10.1. Basic concepts of linear programming. The case of only two independent variables, x_1, x_2, is shown. The linear function ζ, to be minimized, is represented by its contour lines. Nonnegativity constraints require x_1 and x_2 to be positive. Additional constraints may restrict the solution to regions (inequality constraints) or to surfaces of lower dimensionality (equality constraints). Feasible vectors satisfy all constraints. Feasible basic vectors also lie on the boundary of the allowed region. The simplex method steps among feasible basic vectors until the optimal feasible vector is found.

are incompatible, or (ii) there is no minimum, i.e., there is a direction in n-space where one or more of the variables can be taken to infinity while still satisfying the constraints, giving an unbounded value for the objective function. Figure 10.10.1 summarizes some of the terminology thus far.

As you see, the subject of linear programming is surrounded by notational and terminological thickets. Both of these thorny defenses are lovingly cultivated by a coterie of stern acolytes who have devoted themselves to the field. Actually, the basic ideas of linear programming are quite simple. Avoiding the shrubbery, let's elucidate the basics by means of a couple of specific examples; it should then be quite obvious how to generalize.

Why is linear programming so important? (i) Because "nonnegativity" is the usual constraint on any variable x_i that represents the tangible amount of some physical commodity, like guns, butter, dollars, units of vitamin E, food calories, kilowatt hours, mass, etc. Hence equation (10.10.2). (ii) Because one is often interested in additive (linear) limitations or bounds imposed by man or nature: minimum nutritional

requirement, maximum affordable cost, maximum on available labor or capital, minimum tolerable level of voter approval, etc. Hence equations (10.10.3) – (10.10.4). (iii) Because the function that one wants to optimize may be linear, or else may at least be approximated by a linear function — since that is the problem that linear programming *can* solve. Hence equation (10.10.1). For a short, semipopular survey of linear programming applications, see Bland [1].

10.10.1 Fundamental Theorem of Linear Optimization

Imagine that we start with a full n-dimensional space of candidate vectors. Then (in our mind's eye, at least) we carve away the regions that are eliminated in turn by each imposed constraint. Since the constraints are linear, every boundary introduced by this process is a plane, or rather a hyperplane. Equality constraints of the form (10.10.4) force the feasible region onto hyperplanes of smaller dimension, while inequalities simply divide the then-feasible region into allowed and nonallowed pieces.

When all the constraints are imposed, either we are left with some feasible region or else there are no feasible vectors. Since the feasible region is bounded by hyperplanes, it is geometrically a kind of convex polyhedron or simplex (cf. §10.5). If there is a feasible region, can the optimal feasible vector be somewhere in its interior, away from the boundaries? No, because the objective function is linear. This means that it always has a nonzero vector gradient. This, in turn, means that we could always decrease the objective function by running down the gradient until we hit a boundary wall.

The boundary of any geometrical region has one less dimension than its interior. Therefore, we can now run down the gradient projected into the boundary wall until we reach an edge of that wall. We can then run down that edge, and so on, down through whatever number of dimensions, until we finally arrive at a point, a *vertex* of the original simplex. Since this point has all n of its coordinates defined, it must be the solution of n simultaneous *equalities* drawn from the original set of equalities and inequalities (10.10.2) – (10.10.4).

Points that are feasible vectors and that satisfy n of the original constraints as equalities, including the nonnegativity constraints, are termed *feasible basic vectors*. If $n > m$, then a feasible basic vector has *at least* $n - m$ of its components equal to zero, since at least that many of the constraints (10.10.2) will be needed to make up the total of n. Put the other way, *at most* m components of a feasible basic vector are nonzero.

Put together the two preceding paragraphs and you have the *Fundamental Theorem of Linear Optimization*: If an optimal feasible vector exists, then there is a feasible basic vector that is optimal. (Didn't we warn you about the terminological thicket?)

The importance of the fundamental theorem is that it reduces the optimization problem to a "combinatorial" problem, that of determining which n constraints (out of the $m + n$ constraints in 10.10.2 – 10.10.4) should be satisfied by the optimal feasible vector. We have only to keep trying different combinations, and computing the objective function for each trial, until we find the best.

Doing this blindly would take halfway to forever. The *simplex method*, first published by Dantzig in 1948 (see [2]), is a way of organizing the procedure so that (i) a series of combinations is tried for which the objective function decreases at each step, and (ii) the optimal feasible vector is reached after a number of iterations that

is almost always no larger than of order m or n, whichever is larger. An interesting mathematical sidelight is that this second property, although known empirically ever since the simplex method was devised, was not proved to be true until the 1982 work of Stephen Smale. (For a contemporary account, see [3].)

10.10.2 Writing the General Problem in Standard Form

There is a standard form for linear programming problems, and we have to learn how to write a general problem like (10.10.1) – (10.10.4) in this standard form. For definiteness, consider the problem

$$\text{Minimize} \quad \zeta = -40x_1 - 60x_2 \tag{10.10.5}$$

with the x's nonnegative and also with

$$2x_1 + x_2 \le 70 \tag{10.10.6}$$
$$x_1 + x_2 \ge 40 \tag{10.10.7}$$
$$x_1 + 3x_2 = 90 \tag{10.10.8}$$

First, we rewrite the inequalities as equalities. We do this by adding to the problem so-called *slack variables* $x_{n+1}, x_{n+2}, \ldots$ In our example, equations (10.10.6) and (10.10.7) become

$$2x_1 + x_2 + x_3 = 70 \tag{10.10.9}$$
$$-x_1 - x_2 + x_4 = -40 \tag{10.10.10}$$

(A slack variable like x_4 for a $\ge$ inequality is sometimes called a *surplus* variable.) Requiring the slack variables to be nonnegative makes these equalities equivalent to the original inequalities. Once they are introduced, you treat slack variables on an equal footing with the original variables x_i; then, at the very end, you just ignore them. The simplex solution for each slack variable is simply the amount by which the original inequality is satisfied.

The key idea in the simplex method is to start with a feasible basic vector and make a sequence of exchanges between basic and nonbasic variables. At each step the vector stays feasible (satisfies the constraints), and the objective function decreases (or at least does not increase).

How do we find an initial feasible basic vector to start the procedure? Suppose that our example were changed so that equations (10.10.7) and (10.10.8) were both $\le$ inequalities, like (10.10.6). Then, after introducing slack variables, we would have

$$2x_1 + x_2 + x_3 = 70 \tag{10.10.11}$$
$$x_1 + x_2 + x_4 = 40 \tag{10.10.12}$$
$$x_1 + 3x_2 + x_5 = 90 \tag{10.10.13}$$

In this case it is easy to write down a feasible basic vector: Set the original variables x_1 and x_2 to zero and take $(x_3, x_4, x_5) = (70, 40, 90)$. Here $n = 2$ of the constraints, namely $x_1 \ge 0$, $x_2 \ge 0$, are satisfied as equalities, while $m = 3$ components of the feasible basic vector are nonzero. The variables (x_3, x_4, x_5) are called *basic variables*, while the variables that are zero, (x_1, x_2), are called *nonbasic variables*. Note

that if we write equations (10.10.11) – (10.10.13) as a 3×5 matrix equation, then the last three columns of the matrix, corresponding to the slack variables (x_3, x_4, x_5), form a 3×3 unit matrix.

So $\leq$ constraints are easy. But how do we handle constraints like equations (10.10.7) and (10.10.8)? The trick is again to invent new variables called *artificial variables*. We rewrite equation (10.10.8) as

$$x_1 + 3x_2 + x_5 = 90 \tag{10.10.14}$$

Now equations (10.10.9), (10.10.10), and (10.10.14) are almost in the form to give us an easy initial feasible basic vector by setting $x_1 = x_2 = 0$. The obstacle is equation (10.10.10), which would give a negative value for x_4. We have to precede the actual simplex procedure by a preliminary procedure, called *phase one* of the simplex method, to find an initial feasible vector. (The actual optimization is called *phase two*.)

In phase one, we replace our objective function (10.10.5) by a so-called *auxiliary objective function*,

$$\zeta' \equiv -x_4 \tag{10.10.15}$$

We now perform the simplex method on the auxiliary objective function (10.10.15) with the constraints (10.10.9), (10.10.10), and (10.10.14), starting with the basis given by $x_1 = x_2 = 0$. The variable x_4 starts off negative (at -40). Minimizing the function (10.10.15) drives x_4 toward satisfying $x_4 \geq 0$, the condition for feasibility. In fact, we don't even have to solve phase one all the way to the exact minimum. As we do the exchanges between variables during this phase, we continually redefine the auxiliary objective function at each iteration to be minus the sum of all negative basic variables. As soon as all the basic variables are nonnegative, we are done with phase one.

And what if the first phase *doesn't* drive the auxiliary objective function to a negative value (i.e., all basic variables nonnegative)? That signals that there is *no* initial feasible basic vector, i.e., that the constraints given to us are inconsistent among themselves. Report that fact, and you are done.

An artificial variable in an equality constraint is an example of a *zero variable*, a variable that must vanish in the optimal solution. Typically the way a zero variable gets to be zero is by being nonbasic in the optimal solution. So we can precede phase one with a "phase zero" in which we exchange each zero variable out of the basis.

One last piece of jargon: Slack and artificial variables are often called *logical* variables, to distinguish them from the original independent variables, which are sometimes called *structural* variables.

10.10.3 The Simplex Method: A Worked Example

The easiest way to describe the actual simplex procedure is with a worked example. We write the general linear programming problem in the following form: Minimize the objective function

$$\zeta = \mathbf{c} \cdot \mathbf{x} = c_1 x_1 + c_2 x_2 + \cdots + c_n x_n \tag{10.10.16}$$

subject to the constraints

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \tag{10.10.17}$$

and

$$x_i \geq 0, \qquad i = 1, \ldots, n+m \tag{10.10.18}$$

Here we assume that we started with an $m \times n$ matrix of constraint coefficients given by equations like (10.10.3) and (10.10.4). We then added slack variables to the inequality constraints and artificial variables to the equality constraints so that the constraint matrix is now the $m \times (n+m)$ matrix $\mathbf{A}$. The last m columns form an $m \times m$ identity matrix. Note that the coefficients of the slack variables are taken to be $+1$, so that an original $\geq$ inequality will have a negative right-hand side. For our example given in equations (10.10.5) – (10.10.8), transformed as in equations (10.10.9), (10.10.10), and (10.10.14), the matrix $\mathbf{A}$ has five columns:

$$\mathbf{a}_1 = \begin{pmatrix} 2 \\ -1 \\ 1 \end{pmatrix} \quad \mathbf{a}_2 = \begin{pmatrix} 1 \\ -1 \\ 3 \end{pmatrix} \quad \mathbf{a}_3 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \quad \mathbf{a}_4 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \quad \mathbf{a}_5 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \tag{10.10.19}$$

The right-hand side and the objective function coefficients are

$$\mathbf{b} = \begin{pmatrix} 70 \\ -40 \\ 90 \end{pmatrix} \qquad \mathbf{c} = (-40, -60, 0, 0, 0)^T \tag{10.10.20}$$

We partition the matrix $\mathbf{A}$ into two submatrices,

$$\mathbf{A} = \left[\mathbf{A}_B \;\middle|\; \mathbf{A}_N \right] \tag{10.10.21}$$

where we have permuted the columns corresponding to the basic variables to be in $\mathbf{A}_B$, while the nonbasic columns are in $\mathbf{A}_N$. In our example, the initial basic variables are (x_3, x_4, x_5) and the initial basis $\mathbf{A}_B$ is the unit matrix composed of the last three columns of $\mathbf{A}$, $\mathbf{a}_3$, $\mathbf{a}_4$ and $\mathbf{a}_5$. A basic solution of $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ consists of a set of basic and nonbasic variables $[\mathbf{x}_B \mid \mathbf{x}_N]$ with $\mathbf{x}_N = 0$. In our example, initially $\mathbf{x}_N = (x_1, x_2) = 0$. The basic solution satisfies $\mathbf{A}_B \cdot \mathbf{x}_B = \mathbf{b}$, or $\mathbf{x}_B = \mathbf{A}_B^{-1} \cdot \mathbf{b}$.

To derive the simplex method, we need one simple equation: how a basic vector changes as a nonbasic variable (i.e., one that is zero) becomes nonzero. This corresponds to starting at a vertex of the simplex and sliding along an edge toward another vertex. Suppose the variable x_k is the one increasing from zero. The constraint equation $\mathbf{A} \cdot \mathbf{x} = \mathbf{A}_B \cdot \mathbf{x}_B = \mathbf{b}$ becomes

$$\mathbf{A}_B \mathbf{x}'_B + \mathbf{a}_k x_k = \mathbf{b} \tag{10.10.22}$$

since only x_k is nonzero among the nonbasic variables. Multiplying this equation by $\mathbf{A}_B^{-1}$ gives

$$\mathbf{x}'_B = \mathbf{A}_B^{-1} \cdot \mathbf{b} - x_k \mathbf{A}_B^{-1} \cdot \mathbf{a}_k = \mathbf{x}_B - x_k \mathbf{A}_B^{-1} \cdot \mathbf{a}_k \tag{10.10.23}$$

The first application of equation (10.10.23) is to the idea of a *reduced cost*. The coefficient c_i in the objective function (10.10.16) is sometimes called the cost of variable x_i, because it represents the cost of having x_i amount of quantity i in the objective function. The simplex method requires instead the cost of *changing* a variable that is zero (not in the basis) to a nonzero value. If the initial value of the objective function is $\mathbf{c} \cdot \mathbf{x}_B = \mathbf{c}_B \cdot \mathbf{x}_B$ and the final value is $\mathbf{c} \cdot (\mathbf{x}'_B + x_k \mathbf{e}_k)$, where $\mathbf{e}_k$

is a unit vector, then using equation (10.10.23) you find that the difference is $x_k u_k$, where the reduced cost of x_k is given by

$$u_k = c_k - \mathbf{a}_k \cdot \mathbf{y}, \qquad \mathbf{y} \equiv (\mathbf{A}_B^{-1})^T \cdot \mathbf{c}_B \tag{10.10.24}$$

Note that if $u_k < 0$, you can make the value of the objective function smaller by bringing x_k into the basis (making it nonzero).

The simplex procedure consists of the following steps:

1. Find a feasible basis (phase 1)
2. Compute the reduced costs (10.10.24) for all x_k not in the basis.
3. If $u_k \geq 0$ for all k, the solution is optimal: No exchange will improve things. Otherwise, choose k corresponding to the most negative u_k as the entering column.
4. Choose the leaving column i from the *minimum ratio test* (motivated below): Compute
$$\mathbf{x}_B = \mathbf{A}_B^{-1} \cdot \mathbf{b}, \qquad \mathbf{w} = \mathbf{A}_B^{-1} \cdot \mathbf{a}_k \tag{10.10.25}$$
For each component $w_i > 0$, compute the ratio x_B^i / w_i. Choose i that corresponds to the smallest such ratio. (If no $w_i > 0$, the objective function is unbounded. Exit and report this.)
5. Exchange columns i and k and go back to step 2.

The minimum ratio test is the second application of equation (10.10.23), which can be written as

$$(x_B^i)' = x_B^i - x_k w_i \tag{10.10.26}$$

where w_i is defined in equation (10.10.25). For each $w_i > 0$, x_B^i decreases as x_k increases from zero. The minimum ratio test selects the i corresponding to the first x_B^i to hit zero, while the other basis variables are still positive. The idea is to allow x_k to be as big as possible so that the objective function is reduced as much as possible by bringing it into the basis.

Let's see how this applies to our example. We start with phase zero, where we remove the zero variable x_5 from the basis. Suppose we choose x_2 to be the incoming variable (x_1 would work fine, too). Using x_2, we find for the new basis and its inverse

$$\mathbf{A}_B = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & -1 \\ 0 & 0 & 3 \end{pmatrix} \qquad \mathbf{A}_B^{-1} = \begin{pmatrix} 1 & 0 & -\frac{1}{3} \\ 0 & 1 & \frac{1}{3} \\ 0 & 0 & \frac{1}{3} \end{pmatrix} \tag{10.10.27}$$

The new basic solution is

$$\mathbf{x}_B = \begin{pmatrix} x_3 \\ x_4 \\ x_2 \end{pmatrix} = \mathbf{A}_B^{-1} \cdot \mathbf{b} = \begin{pmatrix} 40 \\ -10 \\ 30 \end{pmatrix} \tag{10.10.28}$$

The solution (10.10.28) is not feasible because x_4 is negative. We enter phase one, with $\zeta' = \overline{\mathbf{c}} \cdot \mathbf{x} = -x_4$, i.e.,

$$\overline{\mathbf{c}}_B = \begin{pmatrix} 0 \\ -1 \\ 0 \end{pmatrix} \tag{10.10.29}$$

Here the order of elements corresponds to the order (x_3, x_4, x_2) for the basic variables. We compute the reduced costs from equation (10.10.24). Only $k = 1$ is relevant, since x_5 is never allowed to re-enter the basis (zero variable). We find

$$u_1 = -\mathbf{a}_1 \cdot (\mathbf{A}_B^{-1})^T \cdot \bar{\mathbf{c}}_B = -\tfrac{2}{3} \tag{10.10.30}$$

which is negative, confirming that x_1 should enter. For the minimum ratio test to determine which variable leaves, we need the quantity

$$\mathbf{A}_B^{-1} \cdot \mathbf{a}_1 = \begin{pmatrix} \frac{5}{3} \\ -\frac{2}{3} \\ \frac{1}{3} \end{pmatrix} \tag{10.10.31}$$

So the ratios of the elements in equation (10.10.28) to those in equation (10.10.31) are

$$\frac{40}{5/3} = 24, \qquad \frac{-10}{-2/3} = 15, \qquad \frac{30}{1/3} = 90 \tag{10.10.32}$$

The middle ratio is the minimum, so x_4 goes out. (Note that in phase one we relax the requirement that $w_i > 0$, since we haven't yet made all the variables nonnegative.)

Now the basic variables are (x_3, x_1, x_2). Proceeding as before, we find

$$\mathbf{A}_B = \begin{pmatrix} 1 & 2 & 1 \\ 0 & -1 & -1 \\ 0 & 1 & 3 \end{pmatrix} \qquad \mathbf{A}_B^{-1} = \begin{pmatrix} 1 & \frac{5}{2} & \frac{1}{2} \\ 0 & -\frac{3}{2} & -\frac{1}{2} \\ 0 & \frac{1}{2} & \frac{1}{2} \end{pmatrix} \tag{10.10.33}$$

and

$$\mathbf{x}_B = \begin{pmatrix} x_3 \\ x_1 \\ x_2 \end{pmatrix} = \mathbf{A}_B^{-1} \cdot \mathbf{b} = \begin{pmatrix} 15 \\ 15 \\ 25 \end{pmatrix} \tag{10.10.34}$$

All the variables are positive, so the basis is feasible and we enter phase two, with $\mathbf{c}_B = (0, -40, -60)^T$. We find the reduced cost $u_4 = -30$, so x_4 re-enters the basis. The minimum ratio test (10.10.25) gives a minimum for the term involving x_3, so the next basis is (x_4, x_1, x_2). The basic solution turns out to be $(6, 24, 22)$. When we compute the reduced cost u_3 for this basis, it is positive, so we are done. The minimum occurs at $x_1 = 24$, $x_2 = 22$, and the minimum value, obtained by substitution in the objective function, is -2280. The meaning of $x_4 = 6$ is that the inequality (10.10.7) is satisfied by 6. The other two constraints are satisfied as equalities.

The graphical interpretation of the solution procedure is shown in Figure 10.10.2. The initial basic vector is at the origin. We first proceed to the vertex A, which puts us on the line where the equality (10.10.8) is satisfied. This is not a feasible point, since we are on the wrong side of the line Y. So we move along the line X to the vertex B, which is now feasible. Finally we move to vertex C, which is the minimum value of the objective function.

10.10.4 Degeneracy

Nonbasic variables in a basic feasible solution are all zero. If any *basic* variable is zero, we say the basis is *degenerate*. Geometrically, this situation corresponds in n dimensions to having more than n hyperplanes intersect at a vertex. Degeneracy can cause problems in the simplex method. Consider the simple case when three

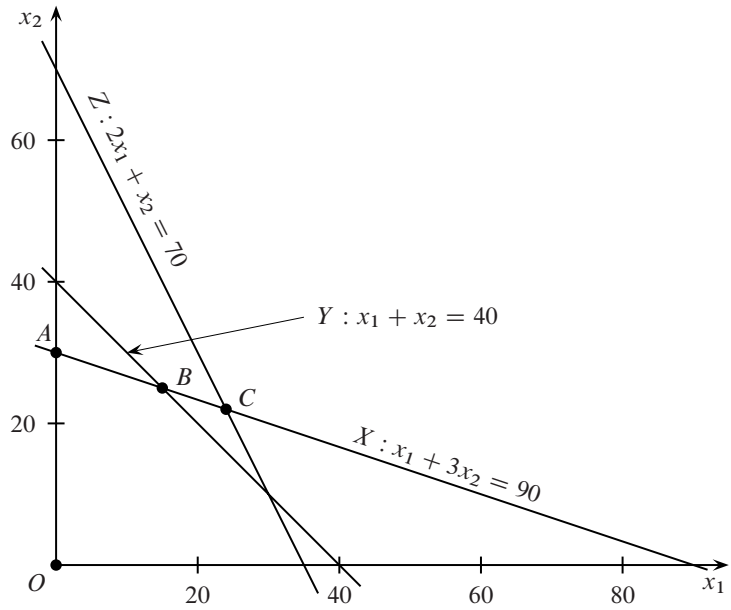

Figure 10.10.2. Graphical interpretation of the simplex solution of the problem (10.10.5) – (10.10.8). The initial basic vector is at the origin O. To satisfy the equality (10.10.8), the first step moves to A, on the line X. This is not yet a feasible point, since it is on the wrong side of the line Y. The next move is to B, which is feasible. We enter phase two, and find that we can reduce the objective function by moving to C. No further moves are possible, so we are done. Note that the figure is really the projection of a five-dimensional simplex onto the x_1-x_2 plane.

lines intersect at a point in two dimensions. Only two of the lines are necessary to define the vertex. When the leaving variable is chosen, it can correspond to the third direction at the vertex. On making this change in the basis, the objective function doesn't improve. You can see this algebraically from equation (10.10.26): If $x_{\boldsymbol{B}}^i = 0$ and $w^i > 0$, then a step size of zero is required for the new variable x_k. Clearly special measures need to be taken.

Degeneracy allows the possibility of *cycling*, where you keep exchanging the same set of vectors in and out of the basis without making any progress. In practice, however, cycling is almost never a problem. More common in very degenerate problems is *stalling*, where you spend a long time making exchanges before finally leaving the vertex.

10.10.5 Sparseness and Stability

If you examine the operations carried out during each simplex step, you see that a key ingredient is to solve equations of the form $\mathbf{A}_{\boldsymbol{B}} \cdot \mathbf{x} = \mathbf{b}$ and similar equations with $\mathbf{A}_{\boldsymbol{B}}^T$. We know that a good method for doing this, absent other considerations, is to use the LU decomposition of $\mathbf{A}_{\boldsymbol{B}}$ (cf. §2.3), since we can use partial pivoting to maintain stability. Decomposing $\mathbf{A}_{\boldsymbol{B}}$ afresh each step is expensive, but since successive bases differ only by the replacement of a single column, one can use techniques analogous to the Sherman-Morrison formula (§2.7) to update $\mathbf{L}$ and $\mathbf{U}$.

However, most linear programming problems that occur in practice have a constraint matrix $\mathbf{A}$ that is very sparse. It is crucial to take advantage of this sparseness in the linear algebra procedures that make up each simplex step, since real-life prob-

lems can involve many thousands or even millions of constraints and variables. Standard LU decomposition with partial pivoting to maintain stability is not desirable, because it leads to excessive *fill-in*, that is, the generation of nonzero matrix elements where there were zeros before. Instead, one chooses the pivot element by a trade-off between stability and sparsity. A popular strategy is based on the *Markowitz criterion* [4]. Here the pivot is a nonzero with the smallest product of the number of other nonzeros in its row and its column. Empirically, the Markowitz criterion works about as well as any other general strategy for minimizing fill-in. In linear programming applications, one also generally needs to impose some kind of stability criterion. In *threshold partial pivoting*, no pivot is less than α times the largest element in its row, where α is a parameter between zero and one. A typical value is $\alpha \sim 0.1$; $\alpha = 1$ gives straight partial pivoting.

The first stable updating procedure for sparse LU decomposition was given by Bartels and Golub [5,6]. This procedure updates $\mathbf{L}$ and $\mathbf{U}$ when a column is replaced in $\mathbf{A}_B$. A good sparse LU algorithm that includes the Bartels-Golub update is that of [7]. It is freely available in the software package LUSOL as part of [8], and we use it in our pedagogical implementation described below.

There are newer methods for the LU decomposition and the update procedure. According to [9], the best of these is probably that of [10].

10.10.6 State-of-the-Art Simplex Codes

A high-quality implementation of the simplex method will have a number of features that we have not discussed so far.

- It will implement more than one variant of the simplex method. In addition to the algorithm we have described, the *primal algorithm*, it will typically also implement the so-called *dual algorithm* (duality is discussed in §10.11). The number of iterations can be significantly reduced by the appropriate use of more than one algorithm.
- It will accept multiple input formats for the specification of the problem, with suitable error checking.
- It will preprocess the problem, with the aim of reducing its size and improving its numerical properties. Many complex problems are inadvertently specified in reducible form.
- It will have multiple options for scaling the problem. As with solving linear equations, no universal scaling algorithm is known for linear programming.
- It will have multiple options for starting the iteration, including several procedures for phase 1.
- It will have several pricing strategies (procedures for selecting the incoming variable). These go by names like multiple pricing, Devex, and steepest edge.
- It will have multiple pivot methods (variants of the ratio test) for the outgoing variable.
- It will handle bounded variables, that is, variables that satisfy a requirement $l_i \le x_i \le u_i$ instead of $x_i \ge 0$. It is possible to handle such bounds using slack variables, but that increases the size of the matrix $\mathbf{A}$. A slight generalization of the algorithm we have described allows you to handle bounded variables directly.
- It will have efficient sparse matrix algorithms.

All these issues are thoroughly discussed in [9].

There are several excellent public domain simplex implementations that incorporate most of the above items. These include CLP [11], GLPK [12] and lp_solve [8]. If you are planning to do any serious LP solving, you should definitely explore these options. It may even be worthwhile to invest in a commercial LP solver. For more information on all these options, see [13]. But first look at the next section, where *interior-point methods* are discussed. It appears now that for many very big problems (but not all), interior-point methods can out-perform simplex methods [14-16]. Even for moderate-sized problems, an interior-point method could be your best choice.

The simplex codes mentioned above are large software efforts, with many thousands of lines of code. They are quite formidable if you want to study how the simplex algorithm works and play around with various options. Accordingly, in a Webnote [17], we give a pedagogical implementation of the simplex method. Even though it uses reasonably good sparse matrix algebra, it is slower than the public domain implementations by one to two orders of magnitude on problems with $\sim 10^4$ variables. If you don't care about the simplex algorithm and you just want a simple method to get your problem solved quickly without getting a public domain code up and running, take a look at our pedagogical interior-point code in the next section.

10.10.7 Other Topics Briefly Mentioned

Problems where the objective function and/or one or more of the constraints are replaced by expressions nonlinear in the variables are called *nonlinear programming problems*. The literature on such problems is vast, but outside our scope. The special case of quadratic expressions is called *quadratic programming*. Optimization problems where the variables take on only integer values are called *integer programming* problems, a special case of *discrete optimization* generally. Section 10.12 looks at a particular kind of discrete optimization problem.

CITED REFERENCES AND FURTHER READING:

Bland, R.G. 1981, "The Allocation of Resources by Linear Programming," *Scientific American*, vol. 244 (June), pp. 126–144.[1]

Dantzig, G.B. 1963, *Linear Programming and Extensions* (Princeton, NJ: Princeton University Press).[2]

Kolata, G. 1982, "Mathematician Solves Simplex Problem," *Science*, vol. 217, p. 39.[3]

Markowitz, H.M. 1957, "The Elimination Form of the Inverse and Its Application to Linear Programming," *Management Science*, vol. 3, pp. 255–269.[4]

Bartels. R.H., and Golub, G.H. 1969, "The Simplex Method of Linear Programming Using the LU Decomposition," *Communications of the ACM*, vol. 12, pp. 266–268.[5]

Bartels, R.H. 1971, "A Stabilization of the Simplex Method," *Numerische Mathematik*, vol. 16, pp. 414–434.[6]

Gill, P.E., Murray, W., Saunders, M.A. and Wright, M.H. 1987, "Maintaining LU Factors of a General Sparse Matrix," *Linear Algebra and Its Applications*, vol. 88/89, pp. 239–270.[7]

lp_solve, `http://groups.yahoo.com/group/lp_solve`.[8]

Maros, I. 2003, *Computational Techniques of the Simplex Method* (Boston: Kluwer).[9]

Suhl, U., and Suhl, L. 1990, "Computing Sparse LU Factorizations for Large-Scale Linear Programming Bases," *ORSA Journal on Computing*, vol. 2, pp. 325–335; 1993, "A Fast LU Update for Linear Programming," *Annals of Operations Research*, vol. 43, pp. 33–47.[10]

CLP, `http://www.coin-or.org/Clp`.[11]